**ANNA UNIVERSITY, CHENNAI-600 025**

**COLLEGE OF ENGINEERING GUINDY**

# A CNN BASED DEEP LEARNING MODEL FOR DETECTING P2P BOTNETS USING FLOW FEATURES

**A PROJECT REPORT**

**Submitted by**

**A T PRANAV VARSHAN 2025165034**

**in partial fulfilment for the award of the degree**

**of**

## MASTER OF ENGINEERING

**IN**

**COMPUTER SCIENCE AND ENGINEERING**

**(DATA SCIENCE AND CYBER SECURITY)**

# A CNN BASED DEEP LEARNING MODEL FOR DETECTING P2P BOTNETS USING FLOW FEATURES

## PSEUDOCODE / ALGORITHM FOR EACH MODULE:

**Module 1: Data Collection:**
Algorithm: Data Collection
Input: Raw network traffic dataset (BoT-IoT)
Output: Structured dataset
1. Download BoT-IoT dataset
2. Extract flow-based features from dataset
3. Store dataset in CSV format
4. Load dataset into system
End

**Module 2: Data Preprocessing:**
Algorithm: Data Preprocessing
Input: Raw dataset
Output: Cleaned and formatted dataset
1. Read dataset
2. Select important features:
('seq',stddev','N_IN_Conn_P_SrcIP','min','state_number','mean','N_IN_Conn_P_DstIP','drate','srate','max')
3. Handle missing values
4. Apply Label Encoding to categorical features
5. Normalize/scale numerical values
6. Convert labels into one-hot encoding
7. Split dataset into training (80%) and testing (20%)
End

**Module 3: Feature Reshaping:**
Algorithm: Feature Reshaping
Input: Preprocessed dataset
Output: CNN-compatible data
1. Convert dataset into NumPy arrays
2. Reshape input into 3D tensor
3. Add extra dimension for CNN input
4. Verify input shape
End

**Module 4: CNN Model Building:**
Algorithm: CNN Model
Input: Training dataset
Output: Trained CNN model
1. Initialize CNN model
2. Add Convolution Layer (64 filters, kernel size = 3)
3. Apply ReLU activation
4. Add MaxPooling layer
5. Flatten output
6. Add Dense layer (32 neurons)
7. Add Output layer with Softmax activation
8. Compile model using:

   - Optimizer: Adam
   - Loss: Categorical Crossentropy
9. Train model using:
   - Epochs = 10
   - Batch size = 32
End

**Module 5: Prediction:**
Algorithm: Prediction
Input: Test dataset
Output: Predicted class labels
1. Load trained model
2. Input test data
3. Generate prediction probabilities
4. Select class with highest probability
5. Map index to category/subcategory
End

**Module 6: Evaluation:**
Algorithm: Evaluation
Input: Predicted results, Actual labels
Output: Performance metrics
1. Calculate Accuracy
2. Calculate Precision
3. Calculate Recall
4. Calculate F1-score
5. Generate Confusion Matrix
6. Plot ROC Curve
End

## 50% PROJECT PROGRESS:

The project titled “CNN-based Deep Learning Model for Detecting P2P Botnets using Flow Features” has been completed up to 50%. The work so far focuses on data preparation and initial model development.
Completed Work

**Dataset Collection:**

The BoT-IoT dataset was successfully loaded using Pandas. It contains both normal and botnet attack traffic.
Feature Selection:
Important flow-based features such as seq, stddev, srate, drate, mean, min, max, and connection-based features were selected.

**Data Preprocessing:**
Label encoding applied to categorical data
Normalization performed for better model performance
Missing values handled

**Dataset Splitting:**
Data was divided into training (80%) and testing (20%) sets.
CNN Model Design:
A 1D CNN model was designed with Conv1D, MaxPooling, Flatten, and Dense layers for classification.

**Work in Progress:**
CNN model training is ongoing
Initial performance evaluation has started

**Next Steps:**
Complete model training
Evaluate using accuracy, precision, recall, and F1-score
Plot confusion matrix and ROC curve
Compare with traditional ML models (KNN, SVM, etc.)

**Summary**
Half of the project is completed, focusing on data processing and model setup. The remaining work involves training, evaluation, and performance improvement.

## Code:

**#Loading Dataset :**

```
import pandas as pd
data=pd.read_csv("UNSW_2018_IoT_Botnet_Final_10_Best.csv",sep=';')
print(data.head())
print(data.shape)
```

```
   Unnamed: 0  pkSeqID proto            saddr  sport            daddr dport  \
0           0        1   tcp  192.168.100.147  49960    192.168.100.7    80
1           1        2   arp    192.168.100.7     -1  192.168.100.147    -1
2           2        3   tcp  192.168.100.147  49962    192.168.100.7    80
3           3        4   tcp  192.168.100.147  49964    192.168.100.7    80
4           4        5   tcp  192.168.100.147  49966    192.168.100.7    80

   seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
0    9  0.068909                 75  0.000000             1  0.068909
1   10  0.000000                  2  0.000131             2  0.000131
2   11  0.064494                 75  0.000000             1  0.064494
3   12  0.064189                 75  0.000000             1  0.064189
4   13  0.063887                 75  0.000000             1  0.063887

   N_IN_Conn_P_DstIP      drate     srate       max  attack category  \
0                 96  14.511893  0.566862  0.137818       1      DoS
1                  1   0.000000  0.000000  0.000131       1      DoS
2                 96  15.505319  0.567549  0.128988       1      DoS
3                 96  15.578993  0.567570  0.128378       1      DoS
4                 96  15.652637  0.567630  0.127774       1      DoS

  subcategory
0        HTTP
1        HTTP
2        HTTP
3        HTTP
4        HTTP
(3668522, 20)
```

**#selecting the features:**

```
Features=['seq',stddev','N_IN_Conn_P_SrcIP','min','state_number','mean','N_IN
_Conn_P_DstIP','drate','srate','max']
X = data[features]
y = data['attack']
print(X.head())
```

```
   seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
0    9  0.068909                 75  0.000000             1  0.068909
1   10  0.000000                  2  0.000131             2  0.000131
2   11  0.064494                 75  0.000000             1  0.064494
3   12  0.064189                 75  0.000000             1  0.064189
4   13  0.063887                 75  0.000000             1  0.063887

   N_IN_Conn_P_DstIP      drate     srate       max
0                 96  14.511893  0.566862  0.137818
1                  1   0.000000  0.000000  0.000131
2                 96  15.505319  0.567549  0.128988
3                 96  15.578993  0.567570  0.128378
4                 96  15.652637  0.567630  0.127774
```

he

print(data.head())

```
   Unnamed: 0  pkSeqID proto            saddr  sport            daddr dport  \
0           0        1   tcp  192.168.100.147  49960    192.168.100.7    80
1           1        2   arp    192.168.100.7     -1  192.168.100.147    -1
2           2        3   tcp  192.168.100.147  49962    192.168.100.7    80
3           3        4   tcp  192.168.100.147  49964    192.168.100.7    80
4           4        5   tcp  192.168.100.147  49966    192.168.100.7    80

   seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
0    9  0.068909                 75  0.000000             1  0.068909
1   10  0.000000                  2  0.000131             2  0.000131
2   11  0.064494                 75  0.000000             1  0.064494
3   12  0.064189                 75  0.000000             1  0.064189
4   13  0.063887                 75  0.000000             1  0.063887

   N_IN_Conn_P_DstIP      drate     srate       max  attack category  \
0                 96  14.511893  0.566862  0.137818       1      DoS
1                  1   0.000000  0.000000  0.000131       1      DoS
2                 96  15.505319  0.567549  0.128988       1      DoS
3                 96  15.578993  0.567570  0.128378       1      DoS
4                 96  15.652637  0.567630  0.127774       1      DoS

  subcategory
0        HTTP
1        HTTP
2        HTTP
3        HTTP
4        HTTP
```

print(data.columns)

```
Index(['Unnamed: 0', 'pkSeqID', 'proto', 'saddr', 'sport', 'daddr', 'dport',
       'seq', 'stddev', 'N_IN_Conn_P_SrcIP', 'min', 'state_number', 'mean',
       'N_IN_Conn_P_DstIP', 'drate', 'srate', 'max', 'attack', 'category',
       'subcategory'],
      dtype='object')
```

**#Encode Data:**

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
data = pd.read_csv(r"UNSW_2018_IoT_Botnet_Final_10_Best.csv",sep=';',
    low_memory=False
)
# Clean column names
data.columns = data.columns.str.strip()
print("Columns:", data.columns)
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = data[col].astype(str)
target_col = None
for col in ['attack', 'category', 'subcategory']:
    if col in data.columns:
        target_col = col
break
if target_col is None:
    target_col = data.columns[-1]
print("Using target column:", target_col)

# Split data
X = data.drop(columns=[target_col], errors='ignore')
y = data[target_col]
if 'Unnamed: 0' in X.columns:
    X = X.drop(['Unnamed: 0'], axis=1)
le = LabelEncoder()
for col in X.columns:
    if X[col].dtype == 'object':
        X[col] = le.fit_transform(X[col])
y = le.fit_transform(y)
print(X.head())
print("Shape:", X.shape)
```

```
Columns: Index(['Unnamed: 0', 'pkSeqID', 'proto', 'saddr', 'sport', 'daddr', 'dport',
       'seq', 'stddev', 'N_IN_Conn_P_SrcIP', 'min', 'state_number', 'mean',
       'N_IN_Conn_P_DstIP', 'drate', 'srate', 'max', 'attack', 'category',
       'subcategory'],
      dtype='object')
Using target column: attack
   pkSeqID  proto  saddr  sport  daddr  dport  seq    stddev  \
0        1      3      1  44407     21   6650    9  0.068909
1        2      0     12      0     10      0   10  0.000000
2        3      3      1  44409     21   6650   11  0.064494
3        4      3      1  44411     21   6650   12  0.064189
4        5      3      1  44413     21   6650   13  0.063887

   N_IN_Conn_P_SrcIP       min  state_number      mean  N_IN_Conn_P_DstIP  \
0                 75  0.000000             1  0.068909                 96
1                  2  0.000131             2  0.000131                  1
2                 75  0.000000             1  0.064494                 96
3                 75  0.000000             1  0.064189                 96
4                 75  0.000000             1  0.063887                 96

        drate     srate       max  category  subcategory
0  14.511893  0.566862  0.137818         1            1
1   0.000000  0.000000  0.000131         1            1
2  15.505319  0.567549  0.128988         1            1
3  15.578993  0.567570  0.128378         1            1
4  15.652637  0.567630  0.127774         1            1
Shape: (3668522, 18)
```

**#Train-Test Split:**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(X_train.shape, X_test.shape)
```

```
(2934817, 18) (733705, 18)
```

**#reshape for CNN:**

```
import numpy as np
X_train = X_train.values
X_test = X_test.values
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
print(X_train.shape)
```

```
(2934817, 18, 1)
```

**#Build CNN Model:**

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense
model = Sequential()
model.add(Conv1D(64, 3, activation='relu', input_shape=(X_train.shape[1],1)))
model.add(MaxPooling1D(2))
model.add(Flatten())
```

```
model.add(Dense(32, activation='relu'))
model.add(Dense(3, activation='softmax'))  # change based on classes
model.summary()
```

```
C:\Users\PRANAV\tf_env\lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argumen
t to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv1d (Conv1D) | (None, 16, 64) | 256 |
| max_pooling1d (MaxPooling1D) | (None, 8, 64) | 0 |
| flatten (Flatten) | (None, 512) | 0 |
| dense (Dense) | (None, 32) | 16,416 |
| dense_1 (Dense) | (None, 3) | 99 |

```
Total params: 16,771 (65.51 KB)

Trainable params: 16,771 (65.51 KB)

Non-trainable params: 0 (0.00 B)
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
```

**#Train Model**

```
data = data.sample(n=50000, random_state=42)
history = model.fit(X_train, y_train, epochs=2, batch_size=256)
```

```
Epoch 1/2
11465/11465 ━━━━━━━━━━━━━━━━━━━━ 47s 4ms/step - accuracy: 0.9998 - loss: 4.2654
Epoch 2/2
11465/11465 ━━━━━━━━━━━━━━━━━━━━ 45s 4ms/step - accuracy: 0.9999 - loss: 0.0274
```

**#test accuracy**

```
loss, acc = model.evaluate(X_test, y_test)
print("Accuracy:", acc)
```

```
22929/22929 ━━━━━━━━━━━━━━━━━━━━ 39s 2ms/step - accuracy: 0.9999 - loss: 0.0012
Accuracy: 0.9998677968978882
```

## DATASET DESCRIPTION

The dataset used for this project is the BoT-IoT dataset, a publicly available cybersecurity dataset developed by UNSW Canberra for detecting botnet attacks in IoT environments.

The dataset contains both normal and malicious network traffic generated in a realistic IoT network environment. It includes multiple types of botnet-related attacks such as:

- Distributed Denial of Service (DDoS)
- Denial of Service (DoS)
- Reconnaissance
- Information Theft

The dataset is originally captured in PCAP format and later converted into flow-based features using network analysis tools like Argus. Each record in the dataset is labeled with attack class, category, and subcategory, which helps in performing both binary and multi-class classification.

For this project, flow-based features are used instead of raw packet data, as they efficiently represent network behavior and reduce computational complexity. The selected features used in this project include:

- seq (Argus sequence number)
- stddev (Standard deviation of flow rate)
- N_IN_Conn_P_SrcIP (Number of incoming connections per source IP)
- N_IN_Conn_P_DstIP (Number of incoming connections per destination IP)
- min (Minimum duration of connection)
- max (Maximum duration of connection)
- mean (Average duration of connection)
- state_number (Encoded flow state)
- srate (Source packet rate)
- drate (Destination packet rate)

These features help in capturing important characteristics such as traffic variation, connection patterns, packet rates, and flow behavior, which are essential for detecting botnet activities.

Due to the very large size of the dataset (several gigabytes), a subset of the dataset is used in this project for efficient training and testing. The subset still contains a large number of records, making it suitable for training deep learning models.

Overall, the BoT-IoT dataset is highly suitable for this project as it provides structured, labeled, and real-world-like network traffic data, enabling effective detection of P2P botnet attacks using deep learning techniques.